

Documentación de Arquitectura — MeritCoin

Formato ARC42 — Versión 0.4.0

Proyecto: MeritCoin — Sistema de Recompensas Académicas Digitales
Institución: Universidad Tecnológica de Bolívar
Fecha: Abril 2026
Estado: En desarrollo activo (Fase 9 de 10 en progreso — Insignias personalizadas)
Rama principal de desarrollo: `main` (fusionada desde `feature/transacciones-base`)

1. Introducción y Objetivos

1.1 Descripción del sistema

MeritCoin es un sistema de incentivos académicos que integra la plataforma LMS Moodle con tecnología blockchain. Permite que los profesores configuren reglas de recompensa por actividad, tipo de actividad o curso, y que los estudiantes acumulen tokens digitales (MRT) e insignias verificables on-chain al completar logros académicos.

El sistema opera de forma híbrida: la lógica de negocio y configuración vive off-chain (Moodle + PostgreSQL), mientras que la emisión de tokens e insignias queda registrada permanentemente on-chain (Ethereum EVM compatible).

1.2 Objetivos de calidad

Prioridad	Atributo	Descripción
1	Integridad	Cada evento académico debe producir exactamente un registro on-chain. Duplicados son rechazados por <code>event_id</code> único (MD5 determinístico de <code>userid+cmid+grade</code>).
2	Trazabilidad	Todo evento queda registrado en la cola Moodle, en el <code>audit_log</code> PostgreSQL y en la blockchain. Tres capas de auditoría independientes.
3	Seguridad	Toda comunicación Moodle → Backend está firmada con HMAC-SHA256. Los contratos usan <code>AccessControl</code> con roles explícitos. Todas las escrituras del plugin usan <code>require_sesskey()</code> .
4	Extensibilidad	El sistema debe poder desplegarse en SAVIO (instancia Moodle de la universidad) con cambios únicamente en la capa de presentación.
5	Operabilidad	El backend debe poder conectarse a cualquier nodo EVM compatible cambiando únicamente la variable <code>BLOCKCHAIN_RPC_URL</code> .

1.3 Partes interesadas (Stakeholders)

Rol	Interés principal
Estudiante	Acumular y consultar monedas e insignias obtenidas en sus cursos; canjear recompensas en el marketplace
Profesor	Configurar reglas de recompensa por curso/actividad sin conocimiento técnico de blockchain; gestionar recompensas canjeables; ver informe de transacciones de su curso
Administrador Moodle	Instalar y configurar el plugin; ver panel global de KPIs, recompensas y canjes; gestionar credenciales del backend
Desarrollador	Mantener y extender el sistema; desplegar en SAVIO

2. Restricciones de Arquitectura

2.1 Restricciones técnicas

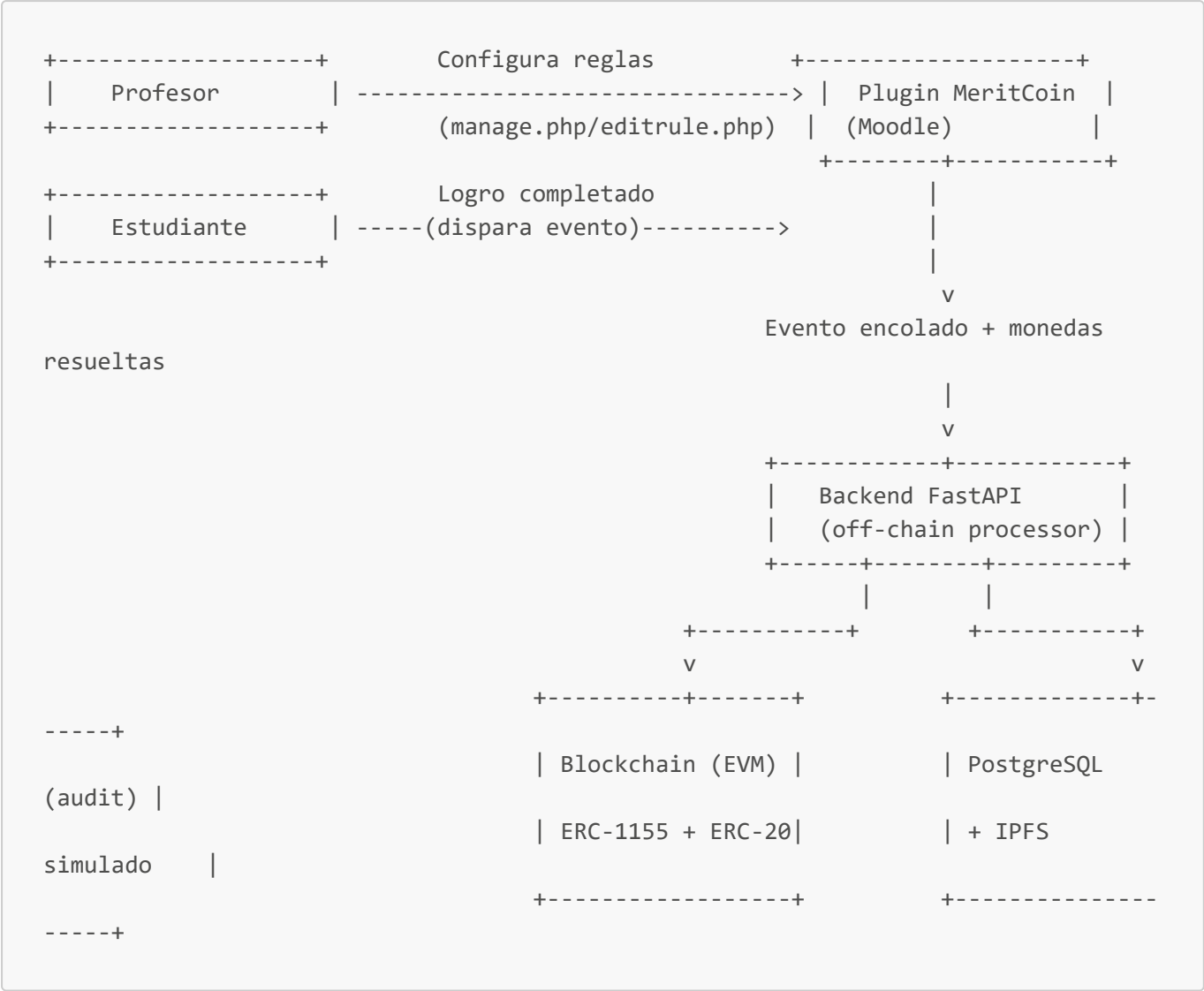
Restricción	Razón
El plugin debe seguir la Moodle Plugin API estándar	Compatibilidad con Moodle 4.x y con SAVIO
Los contratos usan únicamente OpenZeppelin 5.x (sin librerías de pago)	Licencia MIT, auditadas, sin dependencias propietarias
El backend corre dentro de Docker ; el nodo blockchain corre en la máquina host	Restricción de entorno de desarrollo en Windows con Docker Desktop
No se almacenan datos personales on-chain	Privacidad: solo wallets e IDs ofuscados viajan a la blockchain
El backend/.env es la fuente de configuración del backend	config.py lee variables de entorno desde el .env del servicio Docker
El plugin usa file_get_contents (no cURL) para HTTP	El contenedor Bitnami Moodle tiene cURL deshabilitado por defecto

2.2 Restricciones organizacionales

- El proyecto es académico; la infraestructura de producción target es **SAVIO** (Moodle institucional de la UTB).
- El ajuste visual para SAVIO debe poder hacerse sin reescribir lógica de negocio.
- Las claves privadas usadas en desarrollo (**0xac0974...**) son las cuentas públicas de Hardhat; nunca deben usarse en producción.
- El volumen del plugin en **docker-compose.yml** (**./plugin:/bitnami/moodle/local/meritcoin**) debe estar descomentado para que los cambios locales se reflejen en el contenedor.

3. Contexto del Sistema

3.1 Contexto de negocio



3.2 Contexto técnico

Canal	Protocolo	Descripción
Moodle → Backend	HTTP POST + HMAC-SHA256 (file_get_contents)	Envío de eventos académicos firmados
Backend → Blockchain	JSON-RPC (web3.py) via host.docker.internal:8545	Llamadas a <code>mintBadge</code> y <code>mint</code>
Backend → PostgreSQL	asyncpg (SQLAlchemy async)	Persistencia del <code>audit_log</code>
Plugin → MariaDB	Moodle DBAL	Persistencia de queue, rules, earnings, spend, rewards, redemptions
Estudiante → Moodle	HTTPS (navegador)	Dashboard, marketplace, historial de transacciones
Profesor → Moodle	HTTPS (navegador)	Gestión de reglas, recompensas, informe de transacciones del curso

Canal	Protocolo	Descripción
Admin → Moodle	HTTPS (navegador)	Panel global: KPIs, todas las transacciones, gestión de recompensas

4. Estrategia de Solución

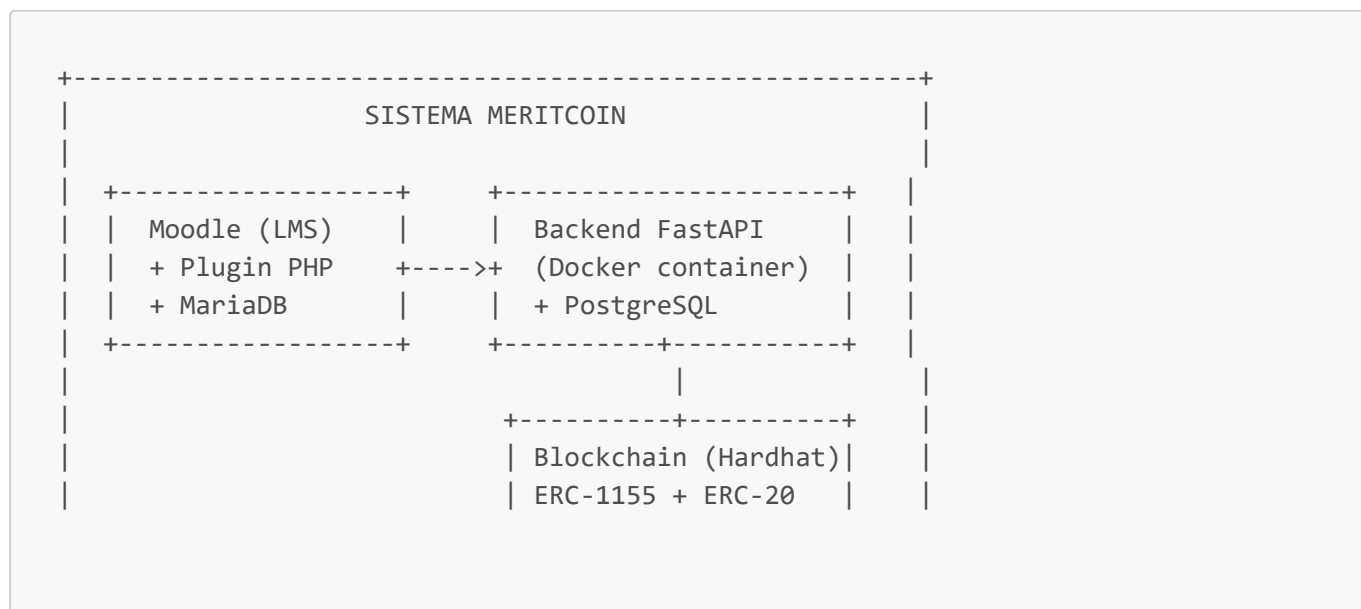
La solución separa en tres capas de responsabilidad claramente delimitadas:

- Capa de configuración y captura (Moodle + Plugin PHP):** El profesor define las reglas de recompensa con tres niveles de granularidad: actividad específica (**activity**), tipo de módulo (**activity_type**) y curso completo (**course**). El observer captura los eventos del LMS, filtra por **itemtype = mod** para ignorar calificaciones globales del curso, aplica las reglas con prioridad jerárquica y calcula el valor de monedas antes de encolar. Esta capa no tiene dependencias directas de la blockchain.
- Capa de procesamiento off-chain (FastAPI + PostgreSQL):** Recibe eventos firmados, verifica integridad HMAC, garantiza idempotencia mediante **event_id** único (MD5 determinístico), genera metadatos OBv2, simula pin IPFS y orquesta las llamadas a la blockchain. Expone endpoints de consulta para el dashboard y el marketplace.
- Capa de registro permanente (Contratos Solidity + EVM):** Emite badges ERC-1155 y tokens ERC-20. Es la fuente de verdad final e inmutable del sistema. El balance real del contrato ERC-20 es consultado por el marketplace para validar canjes.

La decisión de calcular **coins_amount** en el plugin (no en el backend) permite que el backend sea agnóstico a las reglas de negocio del LMS, facilitando la futura integración con otros sistemas distintos de Moodle.

5. Vista de Bloques

5.1 Nivel 1 — Sistema completo



```
|
+-----+
+-----+
```

5.2 Nivel 2 — Plugin Moodle (caja blanca)

Componente	Responsabilidad
<code>observer.php</code>	Escucha <code>mod_completed</code> y <code>grade_item_updated</code> ; filtra <code>itemtype=mod</code> ; genera <code>event_id</code> MD5 determinístico para idempotencia
<code>rules_service.php</code>	Resuelve reglas con prioridad: <code>activity > activity_type > course</code> ; aplica <code>min_grade</code> si está configurado
<code>send_events_task.php</code>	Tarea programada (Moodle Task API) que envía eventos <code>pending</code> al backend vía <code>file_get_contents</code> + HMAC
<code>api_client.php</code>	Encapsula la comunicación HTTP con el backend; genera firma HMAC-SHA256
<code>manage.php</code> + <code>editrule.php</code>	UI del profesor para CRUD de reglas por curso
<code>rule_form.php</code>	Formulario Moodle (Form API) para creación/edición de reglas; incluye dropdown dinámico de módulos del curso
<code>dashboard.php</code>	UI del estudiante: saldo MRT real (del contrato), historial con nombre de actividad y número de reintento, insignias
<code>rewards.php</code>	UI del profesor para crear/gestionar recompensas canjeables del curso
<code>marketplace.php</code>	UI del estudiante para consultar y canjear recompensas; valida saldo por curso (<code>earnings - spend</code>) vs. saldo real del contrato
<code>teacher_transactions.php</code>	Vista del profesor: monedas otorgadas y canjes del curso; KPIs; filtrable por estudiante
<code>admin_marketplace.php</code>	Panel admin: KPIs globales, recompensas, canjes, pestaña “Todas las transacciones” filtrable por curso y estudiante
<code>lib.php</code>	Hooks de navegación: menú global y navegación de curso por rol (estudiante/profesor/admin)
<code>settings.php</code>	Configuración de administrador: URL backend, HMAC secret; registro de páginas externas admin

5.3 Nivel 2 — Backend FastAPI (caja blanca)

Componente	Responsabilidad
<code>api/events.py</code>	Endpoint <code>POST /events/ingest</code> : valida HMAC, delega a <code>events_service</code>
<code>api/students.py</code>	Endpoints de consulta: <code>/balance</code> , <code>/badges</code> , <code>/summary</code>

Componente	Responsabilidad
<code>services/events_service.py</code>	Orquesta el flujo: idempotencia → badges → tokens → audit
<code>services/blockchain.py</code>	Wrapper <code>web3.py</code> : conecta a <code>host.docker.internal:8545</code> , llama <code>mintBadge</code> y <code>mint</code>
<code>services/badges_service.py</code>	Genera metadatos Open Badges v2 y simula pin IPFS
<code>services/tokens_service.py</code>	Calcula y llama mint de tokens ERC-20
<code>services/audit_service.py</code>	Registra resultado final en PostgreSQL
<code>core/config.py</code>	Lee variables de entorno vía <code>pydantic-settings</code>
<code>core/security.py</code>	Verifica firma HMAC-SHA256 de las peticiones entrantes

5.4 Nivel 2 — Contratos Solidity (caja blanca)

Contrato	Estándar	Funciones clave
<code>MeritBadges1155.sol</code>	ERC-1155	<code>mintBadge(address, tokenId, uri)</code> — emite una insignia única por logro
<code>MeritCoinERC20.sol</code>	ERC-20	<code>mint(address, amount)</code> — acuña tokens MRT al wallet del estudiante

Ambos contratos heredan `AccessControl` (roles `ISSUER_ROLE`, `MINTER_ROLE`) y `Pausable` de OpenZeppelin 5.x.

6. Vista de Ejecución

6.1 Escenario principal — Evento de completación/calificación de actividad

Moodle Blockchain	Observer	rules_service	Queue(MariaDB)	Task	Backend
	--grade_item_updated----->				
	--itemtype=mod?				
	--resolve_rules(courseid, userid, cmid, modtype, grade)				
	<-----coins_amount-----				
	--insert(event_id_MD5, coins, pending)----->				

```

| (scheduler cada 1 min) |<--poll-----|
|
| | |---events--->|
|
| | |---POST
/events/ingest+HMAC---> |
| | | | |
verify HMAC | | | |
| | | | |
idempotency? | | | |
| | | | |
mintBadge()-----> | | | |
| | | | |
mint(MRT)-----> | | | |
| | | | |
txHash-----| | | |<--
| | | | |
audit_log | | | |
| | | | |
| | | |<--200 OK-----|
| | | |
| | | |---update(processed)----->|
| | | |
| | | |---insert(earnings: +coins)-->|
|

```

6.2 Escenario — Canje en el marketplace

Estudiante Blockchain	marketplace.php	api_client.php	Backend
	--ver mercado->		
		--GET /summary(wallet)----->	
			--balanceOf(wallet)-----
>			<--balance real-----
		<--balance + badges	
		--calcular saldo disponible	
		(earnings - spend del curso)	
		--mostrar recompensas canjeables-->	
	--canjear----->		
		--validar saldo y stock	
		--insert(redemption)	
		--insert(spend: +precio)	
		--OK: confirmación al estudiante	

6.3 Escenario de idempotencia — Evento duplicado

Si el backend recibe un `event_id` que ya existe en `audit_log`, retorna `200 OK` con "Evento ya fue procesado anteriormente" sin volver a llamar a la blockchain. El plugin marca el evento como `processed`

igualmente.

6.4 Escenario de wallet no registrada

Si el estudiante no tiene wallet configurada en su perfil Moodle, el observer encola el evento con estado `pending_wallet`. La tarea programada ignora estos eventos hasta que el estudiante registre su wallet.

7. Vista de Despliegue

7.1 Entorno de desarrollo (actual)

Máquina host (Windows/Mac/Linux)

```

├─ Docker Desktop
│   ├── meritcoin-backend      (FastAPI, puerto 8000)
│   ├── meritcoin-moodle      (Moodle 4.3, puertos 8080/8443)
│   │   └─ Volumen: ./plugin → /bitnami/moodle/local/meritcoin
│   ├── meritcoin-postgres    (PostgreSQL 16, puerto 5432)
│   └─ meritcoin-mariadb      (MariaDB 10.11, puerto 3306)
└─ Procesos nativos
    └─ npx hardhat node      (puerto 8545)
        └─ scripts/deploy.js → MeritCoinERC20 + MeritBadges1155
  
```

Comunicación Docker → Host: los contenedores usan `host.docker.internal:8545` para alcanzar el nodo Hardhat en la máquina host. La variable `BLOCKCHAIN_RPC_URL=http://host.docker.internal:8545` es obligatoria en `backend/.env`.

Nota crítica sobre el volumen del plugin: la línea `./plugin:/bitnami/moodle/local/meritcoin` en `docker-compose.yml` debe estar descomentada. Si se comenta y se reinicia el servicio, el plugin desaparece de Moodle. Para restaurarlo: descomentar la línea y ejecutar `docker compose up -d --force-recreate moodle`.

7.2 Entorno objetivo — SAVIO (producción)

Servidor UTB

```

├─ SAVIO (Moodle institucional)
│   └─ Plugin local_meritcoin instalado vía zip o directorio
├─ Backend FastAPI
│   └─ Apuntando a nodo EVM de producción (testnet pública o red privada)
└─ Nodo EVM
    └─ Hardhat en modo fork de testnet, o red privada Besu/Geth
  
```


En SAVIO, `BLOCKCHAIN_RPC_URL` apuntará al nodo EVM institucional. El resto de la arquitectura no cambia; únicamente se ajustan variables de entorno y los templates visuales del plugin.

8. Conceptos Transversales

8.1 Seguridad

- **HMAC-SHA256:** cada petición del plugin al backend incluye una firma calculada con `HMAC_SECRET` compartido. El backend rechaza con `401` cualquier petición con firma inválida.
- **Roles en contratos:** `ISSUER_ROLE` controla `mintBadge`; `MINTER_ROLE` controla `mint`. Solo el deployer del backend tiene estos roles asignados.
- **Pausable:** ambos contratos pueden pausarse ante incidentes sin necesidad de redesplicue.
- **sesskey:** todas las acciones de escritura del plugin (crear/editar/borrar reglas, crear/borrar recompensas, canjear) requieren `require_sesskey()` de Moodle para prevenir CSRF.
- **Sin datos personales on-chain:** solo wallets e IDs ofuscados viajan a la blockchain. El `event_id` es un MD5 de `userid+cmid+grade`, no expone información personal.

8.2 Idempotencia

El campo `event_id` en `audit_log` (PostgreSQL) tiene índice único. Si el backend intenta insertar un `event_id` duplicado, la BD lanza una excepción que el servicio captura y convierte en respuesta `200` sin reintentar la transacción blockchain. Adicionalmente, el observer genera un `event_id` determinístico (MD5 de `userid+cmid+grade`), evitando duplicados desde el origen cuando Moodle dispara el mismo evento múltiples veces.

8.3 Trazabilidad en tres capas

Capa	Almacén	Qué registra
Plugin (Moodle)	<code>local_meritcoin_queue</code>	Estado del evento: pending / pending_wallet → processed / failed
Plugin (Moodle)	<code>local_meritcoin_earnings</code>	Ledger de monedas ganadas por usuario y curso
Plugin (Moodle)	<code>local_meritcoin_spend</code>	Ledger de monedas gastadas en canjes por usuario y curso
Backend (off-chain)	PostgreSQL <code>audit_log</code>	<code>event_id</code> , wallet, txHash badge, txHash MRT, CID IPFS, timestamp
Blockchain (on-chain)	EVM	Transacciones inmutables de <code>mintBadge</code> y <code>mint</code>

8.4 Metadatos Open Badges v2 (OBv2)

Cada insignia emitida lleva metadatos conformes al estándar OBv2:

- `name`, `description`, `image` del curso/actividad
- `recipient` (wallet del estudiante, ofuscado con hash SHA-256)

- **issuedOn** (timestamp del evento)
- **verification** (tipo blockchain + dirección del contrato)

El CID IPFS actual es simulado (**QmSimulated...**). En producción se sustituirá por un pin real a nodo IPFS o Pinata.

8.5 Ledger de saldo por curso

El saldo MRT gastable de un estudiante en un curso se calcula en el plugin como:

```
saldo_disponible = SUM(local_meritcoin_earnings.coins_earned WHERE userid,
courseid)
- SUM(local_meritcoin_spend.coins_spent WHERE userid, courseid)
```

Esta lógica es independiente por curso. El marketplace valida adicionalmente que **saldo_disponible** $\geq$ **precio_recompensa** y que el balance real del contrato ERC-20 (consultado al backend) sea también suficiente antes de aprobar el canje.

8.6 Sistema de reglas jerárquico

Las reglas de recompensa tienen tres scopes con prioridad decreciente:

Prioridad	Scope	Descripción
1 (mayor)	activity	Aplica a una actividad específica (cmid concreto)
2	activity_type	Aplica a todos los módulos de un tipo (assign, quiz, forum, etc.)
3 (menor)	course	Aplica al completar el curso entero

Cada regla puede tener un campo **min_grade** opcional: si el evento incluye una calificación inferior al umbral, el evento se descarta sin encolar.

9. Decisiones de Arquitectura (ADR)

ADR-001: Cálculo de monedas en el plugin, no en el backend

Contexto: Las reglas de recompensa son configuradas por el profesor en Moodle. Se evaluó si el cálculo de **coins_amount** debía hacerse en el plugin o en el backend.

Decisión: El plugin resuelve las reglas y envía **coins_amount** ya calculado al backend.

Consecuencias:

- (+) El backend es agnóstico a las reglas de Moodle; puede integrarse con otros LMS.
- (+) Las reglas se pueden cambiar sin redeploy del backend.
- (-) El backend confía en el valor de monedas que envía el plugin; no lo valida de forma independiente.

ADR-002: Comunicación asíncrona Moodle → Backend vía cola interna

Contexto: Los eventos de Moodle ocurren en tiempo real durante la sesión del usuario.

Decisión: El observer encola el evento en MariaDB inmediatamente y la tarea programada lo procesa de forma asíncrona cada minuto.

Consecuencias:

- (+) El usuario no experimenta latencia de la blockchain durante su sesión.
- (+) Si el backend no está disponible, los eventos quedan en cola para reintentar.
- (-) Existe un retardo entre el logro académico y la emisión on-chain (máx. 1 minuto).

ADR-003: Doble base de datos (MariaDB + PostgreSQL)

Contexto: Moodle usa MariaDB de forma nativa; el backend necesita su propia BD.

Decisión: MariaDB exclusivamente para datos del plugin Moodle. PostgreSQL exclusivamente para el audit_log del backend.

Consecuencias:

- (+) Separación de responsabilidades; el backend puede funcionar sin acceso a MariaDB.
- (+) PostgreSQL ofrece mejor soporte para queries analíticas.
- (-) Dos motores de BD en el stack aumentan la complejidad operacional.

ADR-004: IPFS simulado en desarrollo

Contexto: Integrar un nodo IPFS real añade complejidad al entorno de desarrollo.

Decisión: El `badges_service` genera un CID simulado (`QmSimulated...`) en lugar de hacer un pin real.

Consecuencias:

- (+) El entorno de desarrollo es más simple y reproducible.
- (-) Los metadatos OBv2 no son accesibles públicamente en desarrollo; habrá que reemplazar antes del despliegue en SAVIO.

ADR-005: `file_get_contents` en lugar de cURL para HTTP desde el plugin

Contexto: El contenedor Bitnami Moodle tiene la extensión cURL deshabilitada por defecto. El plugin necesita hacer peticiones HTTP al backend.

Decisión: Reemplazar todas las llamadas `curl_*` en `api_client.php` por `file_get_contents` con `stream_context_create`.

Consecuencias:

- (+) Compatible con el entorno Docker de Bitnami sin configuración adicional.
- (+) Código más simple y sin dependencia de extensión PHP.
- (-) `file_get_contents` no ofrece control de timeout tan granular como cURL; en producción se evaluará usar `Guzzle` o habilitar cURL en el contenedor.

ADR-006: Saldo del marketplace basado en ledger local + validación del contrato

Contexto: Se necesitaba decidir si el marketplace usaría el balance on-chain o el ledger local para calcular el saldo gastable del estudiante.

Decisión: El saldo gastable se calcula en el plugin como `earnings - spend` por curso. Adicionalmente, el marketplace consulta el balance real del contrato ERC-20 al backend (`/students/{wallet}/summary`) para validar que el contrato también tiene fondos suficientes.

Consecuencias:

- (+) El saldo por curso es independiente entre cursos (un estudiante puede gastar en curso A sin afectar curso B).
- (+) Se evita aceptar canjes si el minteo on-chain falló silenciosamente.
- (-) Requiere una llamada extra al backend en cada carga del marketplace.

ADR-007: event_id determinístico (MD5) para idempotencia desde el origen

Contexto: Moodle puede disparar el mismo evento múltiples veces (ej. recalificaciones, recargas de página). Se necesitaba evitar múltiples registros en la cola para el mismo evento real.

Decisión: El `event_id` se calcula como `MD5(userid + cmid + grade)` en el observer, antes de insertar en la cola. Se usa `record_exists()` para descartar silenciosamente el evento si ya existe.

Consecuencias:

- (+) Idempotencia garantizada desde el origen, no solo en el backend.
- (+) El backend sigue teniendo su propia capa de idempotencia via `audit_log` (doble protección).
- (-) Si el mismo estudiante mejora su calificación en la misma actividad y obtiene la misma nota, el evento es descartado (caso extremadamente raro, aceptado como trade-off).

10. Esquema de Base de Datos

10.1 MariaDB — Plugin Moodle (v0.4.0)

Tabla	Columnas clave	Propósito
<code>local_meritcoin_queue</code>	<code>userid</code> , <code>courseid</code> , <code>cmid</code> , <code>activity_name</code> , <code>event_id</code> , <code>coins_amount</code> , <code>status</code> , <code>wallet</code>	Cola de eventos pendientes
<code>local_meritcoin_rules</code>	<code>courseid</code> , <code>cmid</code> , <code>rule_scope</code> , <code>mod_type</code> , <code>min_grade</code> , <code>coins_amount</code> , <code>enabled</code>	Reglas de recompensa por curso
<code>local_meritcoin_earnings</code>	<code>userid</code> , <code>courseid</code> , <code>coins_earned</code>	Ledger de monedas ganadas por curso

Tabla	Columnas clave	Propósito
<code>local_meritcoin_spend</code>	userid, courseid, coins_spent	Ledger de monedas gastadas por curso
<code>local_meritcoin_course_config</code>	courseid, coin_name, coin_symbol, contract_address	Config por curso (Fase 9)
<code>local_meritcoin_rewards</code>	courseid, name, description, price, stock, enabled	Recompensas creadas por el profesor
<code>local_meritcoin_redemptions</code>	userid, courseid, rewardid, coins_spent, timecreated	Historial de canjes

10.2 PostgreSQL — Backend (audit)

Tabla	Columnas clave	Propósito
<code>events</code>	event_id (unique), student_wallet, coins_amount, badge_tx, mrt_tx, ipfs_cid, processed_at	Audit log de eventos procesados
<code>audit_log</code>	event_id, action, detail, created_at	Log detallado de operaciones

11. Riesgos y Deuda Técnica

ID	Tipo	Descripción	Impacto	Plan de mitigación
R-01	Riesgo	IPFS simulado en producción invalida la verificabilidad de las insignias OBv2	Alto	Integrar Pinata o nodo IPFS propio antes del despliegue en SAVIO (Fase 10)
R-02	Riesgo	Clave privada del deployer en <code>.env</code> ; si se filtra, un atacante puede mintear tokens arbitrariamente	Alto	Usar HSM o cuenta multisig con Gnosis Safe en producción
R-03	Deuda técnica	Los tests de backend (pytest) no cubren todos los flujos nuevos con <code>rules_service</code> y marketplace	Medio	Revisar y actualizar en la siguiente iteración
R-04	Riesgo	El nodo Hardhat es local; no es un entorno de testnet pública	Medio	Migrar a Sepolia o Polygon Mumbai antes de SAVIO
R-05	Deuda técnica	<code>file_get_contents</code> no permite timeout granular para llamadas al backend	Bajo	Evaluar habilitar cURL en el contenedor Bitnami o usar Guzzle en Fase 10
R-06	Deuda técnica	No existe mecanismo de reintentos explícito para eventos <code>failed</code> en la cola	Bajo	Implementar lógica de retry con backoff en <code>send_events_task.php</code>

ID	Tipo	Descripción	Impacto	Plan de mitigación
R-07	Deuda técnica	Las insignias personalizadas (imagen, nombre, descripción por curso) aún no están implementadas	Medio	En curso — Fase 9 activa
R-08	Riesgo	El volumen del plugin en docker-compose puede quedar comentado accidentalmente al reiniciar Docker	Bajo	Documentado en README; considerar healthcheck que valide el mount

12. Estado del Proyecto

Fase	Descripción	Estado
1	Entorno de desarrollo (Docker)	✅ Completa
2	Contratos inteligentes (Solidity)	✅ Completa
3	Backend FastAPI (Python)	✅ Completa
4	Plugin de Moodle — core (observer, task, queue)	✅ Completa
5	Prueba de flujo completo (E2E)	✅ Completa
6	Gestión de reglas por curso (manage.php, editrule.php, rules_service)	✅ Completa
7	Ledger de ganancias y gasto por curso (earnings, spend)	✅ Completa
8	Dashboard del estudiante + Mercado de recompensas	✅ Completa
9	Insignias personalizadas (imagen, nombre y descripción configurables por curso)	🔄 En progreso
10	Despliegue en SAVIO + ajuste visual al tema de la universidad	📋 Pendiente

Documento generado en Abril 2026 — MeritCoin v0.4.0 — Universidad Tecnológica de Bolívar